Recrd LMSR Prediction Market: Comprehensive Technical Implementation & Flow

1. Architectural Overview & System Lifecycle

In the Recrd prediction market ecosystem, the LMSR (Logarithmic Market Scoring Rule) Automated Market Maker is exclusively used to provide algorithmic liquidity for "long-tail" markets. By design, one binary prediction market is generated for every published video. Since it is impossible to guarantee that human market makers will provide liquidity for thousands of low-volume videos, LMSR mathematically guarantees liquidity at all times.

1.1 The Complete Lifecycle

1. **Event Initialization (Oracle Setup):** When a video is uploaded, the Recrd backend creates a question identifier (e.g., "Will Video #123 reach 10,000 views in 7 days?"). This question is registered with the `UmaCtfAdapter` (the Oracle bridge) which prepares a unique `questionId`.
2. **Condition Preparation (Global Ledger):** The `UmaCtfAdapter` passes the `questionId` to the `ConditionalTokens` contract (the global ERC-1155 ledger). The `ConditionalTokens` contract creates a `conditionId` (a hash of the oracle address and the question). This officially registers the market in the global state.
3. **Pool Deployment & Seeding (Liquidity):** The Recrd backend (or designated liquidity provider) calls `createLMSRMarketMaker` on the `LMSRMarketMakerFactory`. This deploys a brand new, isolated `LMSRMarketMaker` smart contract specifically for this video's market. During deployment, it is seeded with `FKToken` funding (dictating the parameter, or liquidity depth).
4. **User Trading:** Users watching the video predict the outcome by tapping "YES" or "NO". Their `FKToken` is sent to the video's specific `LMSRMarketMaker` contract. The pool calculates the dynamic slippage, pulls the `FKToken`, interacts with the `ConditionalTokens` contract to mint YES/NO tokens, and delivers the requested outcome tokens to the user.
5. **Oracle Resolution:** After 7 days, the UMA Optimistic Oracle is asked for the outcome. Once the dispute period passes and the outcome is verified (e.g., YES it hit 10k views), the `UmaCtfAdapter` formally resolves the market by calling `reportPayouts` on the `ConditionalTokens` contract.
6. **Settlement & Payout:** Trading halts. Users holding the winning tokens call `redeemPositions` on the `ConditionalTokens` contract. The contract burns their winning tokens and releases the underlying `FKToken` collateral at a 1:1 ratio.

---

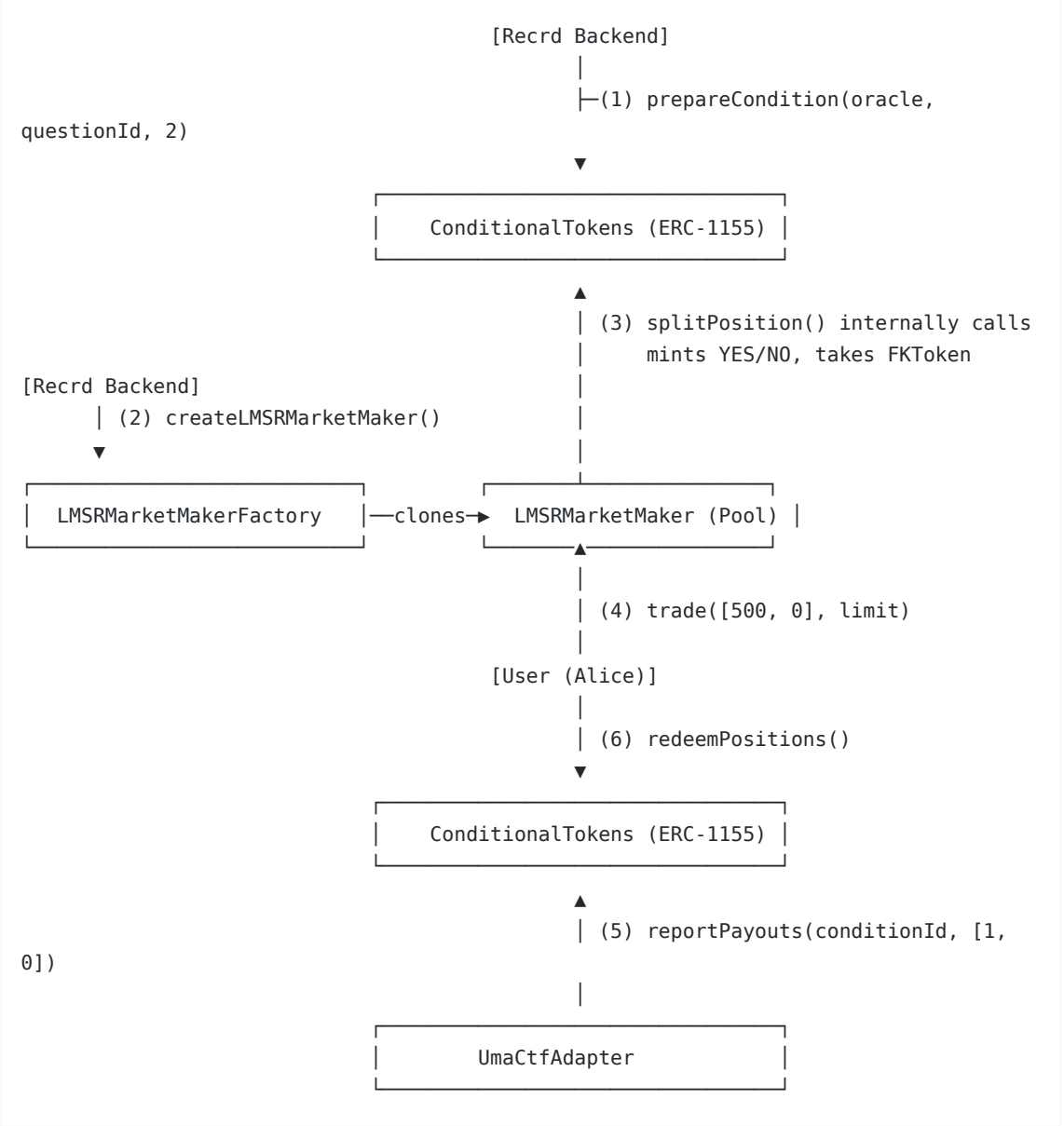
2. Core Smart Contracts & Deep Dive

Contract	Standard	Technical Responsibility
`FKToken`	ERC-20	The fundamental collateral of the platform. Used to fund pools, place bets, and pay out winnings.
`ConditionalTokens`	ERC-1155	The core accounting engine. It stores `payoutNumerators` upon resolution. It is the only contract authorized to mint or burn outcome tokens. It holds all locked `FKToken` collateral in escrow.

`LMSRMarketMakerFactory`	Custom	Clones the LMSR logic using EIP-1167 Minimal Proxies to save gas. Initializes the pool's funding and sets the trading fee (e.g., 2%).
`LMSRMarketMaker`	Custom	The AMM pool. It stores the `b` parameter (funding). It exposes the `trade()` function. It relies on the `Fixed192x64Math` library to accurately compute the logarithmic cost function on-chain.
`UmaCtfAdapter`	Custom	Translates UMA's standard oracle responses into the specific `[1, 0]` or `[0, 1]` array structure required by the `ConditionalTokens` contract.

---

3. Detailed Contract Interaction Diagram



---

4. Technical Example Scenario: Code-Level Execution

Market Context: "Will Video #999 reach 1M views by Friday?" **Collateral:** `FKToken` **Liquidity Parameter ():** 1000 FKToken (Provides moderate slippage for low-volume video markets). **Fee:** 0% (Simplified for this mathematical example).

Phase 1: Pool Deployment and Initialization

The Recrd backend wants to subsidize the market liquidity.

1. The backend approves the `LMSRMarketMakerFactory` to spend 1000 FKToken.
2. It calls:

```
factory.createLMSRMarketMaker(
    pmSystem,           // Address of ConditionalTokens
    collateralToken,    // Address of FKToken
    [conditionId],      // Array of condition IDs (just one for binary)
    0,                  // 0% fee
    whitelist,          // Address 0x0 (public market)
    1000 * 10**18       // 1000 FKToken as 'b' parameter
)
```

3. **Internal Logic:** The factory deploys the pool. The pool transfers 1000 FKToken from the backend to itself. It then calls `pmSystem.splitPosition(...)` which moves the 1000 FKToken into the `ConditionalTokens` contract escrow, and returns 1000 YES and 1000 NO tokens to the pool.
4. **State:** The pool now holds exactly `[1000 YES, 1000 NO]`. It has issued `[0 YES, 0 NO]` to the public.

Phase 2: Alice Trades (Buying 500 YES)

Alice calls `pool.trade([500, 0], 400 * 10**18)`.

1. **Cost Calculation (`calcNetCost`):** The contract uses the LMSR cost formula:
   - `q_old = [0, 0]` -> `C_old = 1000 * ln(2) = 693.14 FKToken`
   - `q_new = [500, 0]` -> `C_new = 1000 * ln( exp(500/1000) + exp(0) ) = 1000 * ln(1.6487 + 1) = 974.07 FKToken`
   - `netCost = C_new - C_old = 280.93 FKToken`
2. **Slippage Check:** The contract verifies `280.93 <= 400` (Alice's `collateralLimit`). The trade passes.
3. **Transfer:** The pool transfers `280.93 FKToken` from Alice to itself.
4. **Minting:** The pool calls `splitPosition` on `ConditionalTokens` with `280.93 FKToken`. `ConditionalTokens` mints `280.93 YES` and `280.93 NO` and sends them to the pool.
   - *Pool Inventory:* Now holds `1280.93 YES` and `1280.93 NO`.
5. **Delivery:** The pool transfers `500 YES` to Alice.
   - *Final Pool Inventory:* `780.93 YES` and `1280.93 NO`.
6. **Price Shift:** The marginal price of YES changes from 0.50 to: `P(YES) = exp(500/1000) / (exp(500/1000) + 1) = 1.6487 / 2.6487 = 0.622 FKToken/YES`

Phase 3: Alice Sells 200 YES (Taking Profits Early)

A few days later, the video is performing well. The price of YES has risen to `0.80 FKToken`. Alice decides to lock in partial profits before the market officially closes. She does not need to find a buyer—the LMSR AMM mathematically acts as the buyer of last resort.

1. **Action:** Alice calls `pool.trade([-200, 0], 150 * 10**18)`.
   - The negative `-200` means she is *selling* 200 YES tokens back to the pool.
   - The `150` acts as a `collateralLimit` in reverse—it is the *minimum acceptable payout* she expects to receive in FKToken.
2. **Cost Calculation (`calcNetCost`):**
   - The pool calculates the cost difference. Because she is returning YES tokens, the pool's `q_yes` liability is decreasing.
   - When q decreases, the cost function C(q) outputs a *negative* number.
   - Assume the math computes a payout of `-155 FKToken`. A negative cost means the pool *owes Alice* 155 FKToken.
3. **Slippage Check:** The pool verifies that `155 >= 150` (Alice's minimum limit). The trade passes.
4. **Execution:**
   - Alice transfers `200 YES` back to the pool.
   - The pool transfers `155 FKToken` from its own collateral reserves back to Alice's wallet.
   - (If the pool's raw collateral runs low, it calls `pmSystem.mergePositions()` to burn its YES and NO pairs and retrieve more FKToken from the global escrow).
5. **Price Rebalancing:** By absorbing Alice's sold YES tokens, the pool's `q_yes` liability decreases. The LMSR formula interprets this as reduced demand, so the marginal price of YES automatically drops slightly (e.g., from 0.80 back to 0.78), preparing a fair price for the next trader. Alice has successfully taken early profits.

Phase 4: Oracle Resolution

The video reaches 1.2M views on Friday.

1. The UMA Optimistic Oracle dispute window closes with the result: `YES`.
2. The `UmaCtfAdapter` executes:

```
pmSystem.reportPayouts(questionId, [1, 0]); // Index 0 is YES, Index 1 is NO
```

3. **State Change:** The `ConditionalTokens` contract permanently maps this `conditionId` to a payout numerator of `1` for YES and `0` for NO.

Phase 4: Settlement and Redemption

Alice sees she won and wants to claim her payout.

1. Alice calls directly into the global ledger:

```
pmSystem.redeemPositions(
    FKToken,         // The collateral token to retrieve
    parentCollectionId, // 0x0 for base conditions
    conditionId,
    [1, 2]           // The index sets for YES (1) and NO (2)
)
```

2. **Burn & Release:** The `ConditionalTokens` contract looks at Alice's balance. It sees she has 500 YES. Since the payout for YES is `1`, it burns her 500 YES tokens and transfers exactly `500 FKToken` from its locked escrow back to Alice's wallet.
3. **Financial Outcome:**
    - Alice spent `280.93 FKToken`.
    - Alice received `500 FKToken`.
    - Alice's net profit: `+219.07 FKToken`.

---

5. System Design Considerations for Recrd

1. **Funding Subsidies & The Parameter (Liquidity vs. Risk):** LMSR pools require initial capital to operate. When Recrd deploys a new pool, it must seed it by locking `FKToken` into the contract. This directly establishes the ** parameter**.

    What is the parameter? It dictates the "liquidity depth" of the market. It controls how many tokens a user can buy before the price shifts dramatically.

    The Maximum Bounded Loss (): The genius of LMSR is that the pool's losses are mathematically capped. If everyone buys YES, the price curves to 0.999 FKToken. At this price, buyers are paying 1 FKToken to win 1 FKToken, so the pool stops losing money. The absolute maximum the pool can ever lose is .

    If b=1000, the maximum loss is `1000 * ln(2) ≈ 693 FKToken`. Recrd must view this 693 FKToken as a maximum possible "liquidity acquisition cost" per video. If users bet incorrectly (or if trades go both ways), the pool's loss decreases and it can even turn a profit via trading fees.

    How to Tune the Dial for Recrd:

    - **High (e.g., 10,000):** Use this for massive influencers. It provides **deep liquidity**—whales can buy thousands of tokens without the price spiking wildly, creating a fantastic user experience. The trade-off is a high maximum bounded loss (FKToken), so Recrd takes on more risk.
    - **Low (e.g., 100):** Use this for small creators. It severely limits Recrd's financial risk (max loss FKToken). However, it creates **shallow liquidity**. If a user tries to buy just 50 YES tokens, the price will instantly spike from 0.50 to 0.90 (high slippage), resulting in a poor user experience.

2. **ERC-1155 Interface:** The frontend MUST be configured to read ERC-1155 token balances to show users their portfolio, rather than standard ERC-20 balances. The `tokenId` is deterministically generated by hashing the `collateralToken`, `conditionId`, and outcome index set.

3. **Batch Redemptions:** Users who bet on multiple videos do not need to redeem one by one. `ConditionalTokens` supports batch redemption, allowing users to burn multiple winning `tokenIds` and withdraw all accumulated `FKToken` in a single, gas-efficient transaction.